ASTANA IT UNIVERSITY

draWeb
PROBATIONARY PERIOD 1
CONSOLIDATED PROJECT REPORT

Development of a Visual Web Application Design and Code Generation System Based on Artificial Intelligence

Educational Programme: Software Engineering

Team: Yin&Yang

Students: Rassul Tokatov (SE-2419); Zholdubayev Diyar (SE-2418)

Supervisor: Praveen Kumar

Reporting period: October 2026

Repository: https://github.com/Dirolka/draWeb

YouTrack: https://cybertora.youtrack.cloud/projects/DRA

Table of Contents

1. Project Proposal

2. Evidence of Relevance

3. Literature Review

4. Existing System Comparison

5. Technology Comparison

6. Requirements Specification

7. Architecture and Diagrams

8. UX / Prototype

9. Verification and Evaluation Plan

10. Project Management and Risks

11. Supervisor Feedback

1. Project Proposal

Development of a Visual Web Application Design and Code Generation System Based on Artificial Intelligence (draWeb)

Project Name:	**draWeb**
Status:	Planning and visual mockup stage
Educational Programme:	Software Engineering

Project Name:	**draWeb**
Team:	Yin&Yang
Team Members (developers):	Rassul Tokatov, Zholdubayev Diyar
Academic Groups:	SE-2419, SE-2418
Supervisor:	Praveen Kumar
Date:	18 October 2026
Project Type:	Team-based software engineering diploma project

1.1. Problem Statement

Building functional web applications often requires expertise in interface design, frontend development, backend development, and integration. Individuals who can describe or sketch a desired interface but do not possess sufficient programming skills may struggle to turn that idea into working software. The proposed draWeb system aims to reduce this gap by combining visual design tools with AI-assisted source-code generation. This problem statement is a project hypothesis and must be validated through user interviews.

1.2. Target Users

The proposed target groups are students, web designers, small-business owners, and beginner developers. Their workflows and needs will be verified in 3–5 discussions with representative users as required by the project preparation guidelines.

1.3. Aim

To design and develop a visual web application authoring system that interprets user-created layouts and generates editable frontend and backend code using artificial intelligence.

1.4. Objectives

- Analyze user needs and compare existing visual design and AI website-building tools.
- Design a visual canvas supporting freehand drawing, drag-and-drop blocks, and uploaded visual assets.
- Design an AI pipeline that interprets the visual layout and produces frontend and backend code.
- Provide preview and iterative text-based editing of the generated website.
- Design mechanisms for account authentication, project saving, version history, responsive layouts, and ZIP/source-code export.
- Define and implement test cases to evaluate layout fidelity, code quality, functional behavior, and usability.

1.5. Proposed Solution and Main User Journey

A user signs in, creates a project, sketches an interface on a canvas or assembles it from ready-made components, and may upload images, icons, or animations. The user requests generation; an AI-based service interprets the design and produces frontend and backend code. The user reviews a preview, requests corrections through text prompts, saves revisions, and exports the source code or a ZIP archive. Deployment to a public URL is not yet included in the confirmed scope.

1.6. Expected Outcomes

- An interactive design canvas and component-based interface editor.
- A prototype-to-code generation workflow targeting a functional web application, including frontend and backend components.
- Website preview, prompt-based revisions, project persistence, version history, and source/ZIP export.
- Responsive behavior for mobile displays, subject to defined acceptance criteria.

- Technical documentation, architecture diagrams, testing evidence, and a traceable Git/YouTrack history.

1.7. Scope and Boundaries

In scope (planned): visual canvas, movable blocks, asset uploads, AI-assisted generation, code preview/export, user authentication, saving, history, and responsive layouts.

Out of scope / undecided: one-click hosting, purchased domains, and guaranteed generation of arbitrary complex production backends. The first demonstrable version will focus on a limited set of common website components and selected backend scenarios; the exact supported feature set will be agreed with the supervisor.

1.8. Preliminary Technology Stack

The team is considering Vue.js for the frontend; JavaScript/Node.js and Go for backend services; PostgreSQL for structured data; Supabase Auth for authentication; OpenAI API for AI-assisted interpretation and code generation; and Docker for packaging. A virtual private cloud (VPC) may be used for infrastructure isolation. These are proposed choices, not completed integrations, and will be justified through a technology comparison.

1.9. Engineering Complexity and Evaluation

Key challenges include mapping ambiguous sketches to structured interface components, generating interoperable frontend/backend code, keeping edits consistent across revisions, securing generated code, and evaluating reliability.

Candidate metrics include task completion, similarity between an input design and the rendered output, successful build rate, functional test pass rate, correction cycles, and task time compared with a baseline workflow. Measurement protocols and thresholds will be specified in the verification plan.

1.10. Team Roles and Current Status

The two students intend to share the overall workload approximately equally across design, frontend, backend, AI integration, testing, and documentation. For formal assessment, each task will have one named owner in YouTrack and individual Git commits or reviewed pull requests. At the time of the first project proposal, only initial mockup material was documented. Subsequent Phase 1 prototype implementation is described in Sections 7–10.

1.11. Initial Milestones and Risks

- Requirements and validation: user interviews, project proposal, literature and competitor review.
- Design: architectural diagrams, mockups, and testable acceptance criteria.
- Implementation: canvas, code generation, revision workflow, storage and authentication.
- Evaluation: usability tasks, code compilation and functional tests; documentation and defense preparation.

Key risks: output-code correctness, model cost and latency, unsafe generated code, expanded scope, and coordination across two backend stacks. Mitigation: limited MVP scenarios, incremental testing, supervised milestones, and documented technology choices.

1.12. Open Items for Supervisor Review

- Agree on MVP boundaries and supported generated backend functionality.
- Verify the technology split between Node.js and Go and whether a VPC is required.
- Review the mockup video and confirm the technical scope and originality claims.

- Agree on a schedule, individual responsibilities, and evaluation thresholds.

2. Evidence of Relevance

Official Diploma Topic:	**Development of a Visual Web Application Design and Code Generation System Based on Artificial Intelligence**
Product:	draWeb
Evidence Examined:	Two screenshot-based chat interviews, one recorded Google Meet session, and its 5-page text transcript.
Status:	Three substantive interviews documented: two written chats and one recorded meeting with a supplied transcript. Consent to publish identifiable material is not confirmed.
Date:	19 October 2026

2.1. Purpose and Research Method

The purpose of this qualitative study is to assess problems faced by potential users in creating and maintaining websites, and to derive preliminary requirements for draWeb. Evidence comprises two semi-structured conversations documented in screenshots and a separate six-minute Google Meet recording. The written exchanges contain substantive answers about prior experience, required functionality, product adoption and concerns. The recorded online meeting has a supplied five-page transcript. The analysis distinguishes participant responses from product claims.

Participants are coded to avoid publishing their names or account details. Interviewers' descriptions of draWeb are treated as product claims, not demonstrations or independently verified technical capabilities. This is a small convenience sample; it does not establish population-level demand.

2.2. Recruitment and Evidence Register

ID	Participant Category	Format	Evidence / Verification
INT-01	Student and small-business operator	Messaging interview	Four supplied chat screenshots; substantial responses documented; consent to use anonymised quotations should be confirmed.
INT-02	Clothing seller / small-business operator	Messaging interview	Five supplied chat screenshots; substantive responses documented; consent to use anonymised quotations should be confirmed.
INT-03	Potential user interested in IELTS / English-learning website; no previous website-building experience	Google Meet video	Uploaded MP4 and 5-page interview transcript; content and recording permission documented. Publication consent and date require verification.

2.3. Interview INT-01 — Student with Small Business

Topic	Evidence-Grounded Finding
Profile / context	Participant states they are a student who also runs a small business; they have not created websites independently due to not knowing programming languages.
Previous tools	Has tried Tilda for a simple page. Stated that complex requirements such as user registration, relational database integration, and a personal account section become excessively difficult.
Pain points	High cost of hiring professional developers; limited available time to learn HTML, CSS, and JavaScript; difficulty adding advanced functional modules independently.
Desired functionality	AI-generated functional website; visual editor for independent ongoing content modifications; hosting and publishing capability; product catalogue, customer registration, and order management.
Priority / concerns	AI generation, visual editor, and publishing ranked as the top three priorities; concerns raised regarding registration reliability, personal data protection, runtime errors, and AI misinterpreting technical prompts.

Topic	Evidence-Grounded Finding
Adoption / pricing	Wants to evaluate a limited free tier first; willing to consider paying for premium tiers if high software quality and tangible time savings are verified.
Suggestions	Straightforward interface, pre-built domain templates, instant preview, and downloadable source code archives for potential handover to developers.
Evidence limitations	Conducted exclusively as a text interview. No independent technical hands-on test, runtime telemetry, or binding commercial commitment.

2.4. Interview INT-02 — Clothing Seller

Topic	Evidence-Grounded Finding
Profile / context	Participant sells apparel via Instagram and currently processes inbound orders through Direct messaging and WhatsApp; seeks a dedicated online e-commerce platform.
Previous experience	Commissioned an initial product showcase website from an external freelancer. Subsequent modifications required re-engaging the contractor, incurring delays of several days and recurring fees.
Required website features	Shopping cart, customer authorization, user profile dashboard, checkout workflow, and an administrative panel for order tracking, inventory, and price updates.
Pain points	Critical dependency on third-party developers for minor content revisions; unpredictable turnaround times and costs; inability to configure servers and databases without external assistance.
Desired workflow	User describes website parameters via natural language, receives an executable baseline, and independently updates promo banners, product collections, and page layouts without writing code.
Security / quality concerns	Application instability, complex technical deployment, customer payment data vulnerability, and inconsistent mobile layout responsiveness.
Onboarding / pricing	Prominent «Create website» call to action, interactive onboarding templates, and quick guides; prefers exploring a trial version before purchase; hypothetically estimated KZT 5,000–10,000 per month conditional on replacing developer expenditure. This represents subjective

Topic	Evidence-Grounded Finding
	willingness to pay, not market validation.
Special request	Ability to reconfigure functional layout components via natural-language instructions (e.g., adding an item category or resizing primary action buttons).
Evidence limitations	Self-reported user preferences rather than documented task completion; the participant was not observed operating the draWeb platform during this session.

2.5. Interview INT-03 — Recorded Session

The recorded Google Meet session is corroborated by a five-page transcript in Russian («Meet – vtx-zmta-ppy.docx»). The participant, designated INT-03, is addressed as Marlen. Explicit verbal consent to record was requested by the interviewer at 00:00:32 and granted at 00:00:36. The respondent had previously considered developing an interactive educational platform for English language learning and IELTS examination preparation combining theory with practice, but was unable to execute the concept due to lack of technical background. Although familiar with the conceptual category of tools such as Figma and Lovable, the participant had no practical experience with them.

The participant expressed strong interest in obtaining an end-to-end operational software implementation rather than purely static visual mockups, and positively assessed conversational design refinement upon explanation. The respondent placed primary importance on interface simplicity for non-programmers and articulated apprehension regarding the inability to audit or verify source code generated by external automated engines. During the prototype demonstration, the participant stated that the workflow was intuitive and reported no usability friction during the session. These findings constitute qualitative user sentiment rather than verified system readiness.

Evidence references: supplied video file (Meet – vtx-zmta-ppy.mp4) and transcript (Meet – vtx-zmta-ppy.docx), specifically timestamps 00:00:32–00:00:36 (recording consent), 00:00:46–00:01:53 (prior experience), 00:02:21–00:03:27 (full implementation and conversational refinement), 00:03:46–00:04:40 (usability and verification concerns), and 00:05:06–00:05:30 (prototype reaction). Consent for public dissemination of personal identifiers, video records, or verbatim remarks has not been established and requires separate formal authorization.

2.6. Findings Synthesis

ID	Finding	Evidence	Implication for draWeb
F-01	Website maintenance without coding is a persistent user requirement.	INT-01, INT-02, INT-03 (interest in natural-language refinement)	Prioritize visual manipulation of existing layouts, typography, graphical assets, and content blocks.
F-02	Users require functional backend capabilities beyond presentation-tier design.	INT-01, INT-02	Define constrained full-stack starter templates (e.g., product lists, authentication, forms, order persistence); avoid arbitrary unconstrained backend generation.
F-03	Low onboarding friction is critical for user adoption.	INT-01, INT-02, INT-03 (emphasis on accessible UI)	Implement guided project initialization, template selections, and immediate rendering preview.
F-04	Security, operational reliability, and code validity constitute major adoption obstacles.	INT-01, INT-02 (data security and crash prevention); INT-03 (trust and verifiability)	Establish automated test suites, credential safeguards, transparent previews, and verifiable architectural summaries.
F-05	Users expect trial access prior to commercial commitment.	INT-01, INT-02	Provide a free exploration sandbox; commercial billing remains outside initial academic scope.
F-06	Conversational prompt-based editing provides high perceived	INT-02, INT-03 explicitly; INT-01	Prototype conversational AI patch generation and verify

ID	Finding	Evidence	Implication for draWeb
	utility.	values no-code modifications	that subsequent revisions do not break existing components.
F-07	Direct demand for freehand sketch-to-code workflows is not yet empirically confirmed.	Neither chat conversation established an established sketch-based habit	Conduct targeted comparative usability testing contrasting freehand sketching against structured drag-and-drop palettes.
F-08	Non-technical users demonstrate skepticism toward uninspectable black-box code.	INT-03	Deliver transparent preview containers, fully exportable source archives, and understandable technical status summaries.

2.7. Assumptions Versus Verified Observations

Claim	Evidence Status
Small businesses face high costs and friction paying developers for routine site updates	Supported by INT-02; corroborated by budget concerns raised by INT-01.
Target users require full-stack features rather than merely static visual landing pages	Supported across text interviews via explicit feature requests (accounts, databases, carts).
Users fundamentally prefer hand-drawing interfaces over prompt inputs or component libraries	Not established; requires empirical usability benchmarking.
draWeb reliably synthesizes production-grade client and server code	Not established by qualitative interviews; must be verified via automated technical test suites.
The product possesses general commercial market viability	Not established; a three-participant convenience sample does not warrant statistical generalization.
Three qualitative interview records have been empirically documented and verified	Verified; substantive written logs and an audio-video session with transcript are recorded.

2.8. Proposed Requirements Derived from Interviews

Req ID	Proposed Requirement	Source
FR-01	Enable direct visual editing of synthesized textual content, layouts, images, and structural blocks without programming.	F-01
FR-02	Provide a constrained set of full-stack starter templates encompassing user authentication, catalog displays, and form submissions.	F-02
FR-03	Incorporate interactive real-time previews and a step-by-step guided project setup wizard.	F-03
FR-04	Support natural-language instruction prompts to modify layout elements with visible diff confirmation.	F-06
NFR-01	Enforce strict account security, credential separation, and data isolation across project workspaces.	F-04
NFR-02	Ensure consistent layout responsiveness across standard mobile and desktop viewport profiles.	F-04
NFR-03	Expose explainable system states, validation logs, and fully downloadable unbundled source code.	F-08
VAL-01	Conduct empirical usability evaluation comparing user task efficiency between freehand sketching and modular block composition.	F-07

2.9. Evidence Management and Ethics

Original chat screenshots, video files, and transcripts must be preserved in a secure private repository accessible exclusively to the project team and academic advisors. Public dissemination of unredacted personal names, contact credentials, telephone numbers, or unconsented video recordings is strictly prohibited. Paraphrased English summaries are utilized to protect participant anonymity. Explicit formal consent must be obtained prior to disclosing any primary audio-visual records or verbatim statements.

2.10. Conclusion and Next Actions

Qualitative analysis across three candidate interviews indicates clear user demand for no-code website modification, bounded full-stack workflows, and intuitive guided setup. However, the data does not confirm universal demand for hand-drawn sketching over component assembly, nor does it establish production robustness of the AI engine.

Prior to submission, the team must: (1) verify and freeze original conversation dates; (2) secure written consent for anonymized citation usage; (3) safeguard primary media in private archives; (4) review the derived functional requirements with the academic supervisor; and (5) document engineering refinements integrated into draWeb.

Appendix A. INT-03 — Verifiable Interview Record

Field	**Evidence from Meeting Transcript**
Participant	INT-03; interested in developing an IELTS/English-learning web resource; addressed as Marlen.
Method	Recorded Google Meet video session; supported by full Russian-language transcript.
Consent	Verbal consent to record granted at timestamps 00:00:32–00:00:36. No public release consent granted.
Previous experience	No prior web development experience; conceptualized educational project but lacked technical skillset (00:00:46–00:01:53).
Current tools	Aware of contemporary AI and visual website design tools, but had not utilized them (00:01:46–00:01:53).
Challenges	Unclear structural path for creating test modules and interactive exercises; uncertainty regarding developer procurement (00:01:20).
Requested capabilities	Turnkey full-stack application synthesis; natural-language layout adjustment; beginner-accessible UI (00:02:21–00:03:46).

Field	Evidence from Meeting Transcript
Risks / doubts	Inability to inspect, audit, or verify generated application code delivered by black-box systems (00:04:13–00:04:40).
Prototype reaction	Stated interface was immediately clear; reported no noticeable friction during demonstration (00:05:06–00:05:30).
Evidence limitation	Subjective feedback; does not establish technical scalability or full-stack software correctness.

Transcript reference: Meet – vtx-zmta-ppy.docx

3. Literature Review

3.1. Scope and Research Questions

The literature review investigates: (RQ1) how graphical user interfaces can be translated into web code; (RQ2) how visual fidelity and functional correctness can be evaluated; and (RQ3) how iterative AI feedback and code-level verification can improve reliability. The proposed draWeb system combines a drawing canvas, drag-and-drop blocks, asset uploads, text-based refinement, preview, and export of frontend and selected backend code. These are intended capabilities, not yet verified implementation results.

3.2. Critical Synthesis

Earlier screenshot-to-code research demonstrated the feasibility of mapping interface images to markup [S1], but did not establish reliable full-stack functionality. A more recent real-world benchmark [S2] reveals weaknesses in visual element recall and layout reconstruction. Larger-scale evaluation work [S3] emphasizes layout and style consistency. Thus, visual resemblance must be tested separately from working application behavior.

Software engineering benchmarks [S4–S6] demonstrate that generated code cannot be judged solely by its plausibility: tests, reproducible execution and realistic

tasks are necessary. Iterative refinement research [S7–S8] motivates draWeb's planned chat-based change cycle, although findings from general reasoning tasks should not be treated as proof of improvement on web projects.

Research gap and project positioning: an integrated, beginner-friendly workflow in which a user draws or composes a layout, requests limited supported functionality, previews generated output, edits it conversationally, and exports inspectable frontend/backend source. This is a proposed combination and workflow emphasis, not a claim that competitors lack visual editing, AI generation or backend support.

3.3. Structured Analysis of Eight Verified Scholarly Research Works

ID / Reference	Approach / Findings	Critical Limitation	Relevance to draWeb
S1 Beltramelli (2017), pix2code	Screenshot → UI code	Early feasibility; dated architectures and no full-stack validation	Image-to-layout pipeline
S2 Si et al. (2024), Design2Code	Real-page screenshot-to-code benchmark	Visual recall/layout inaccuracies remain	Screenshot fidelity evaluation
S3 Lai et al. (2025), WebRenderBench	Layout/style consistency benchmark	Rendering fidelity does not establish backend correctness	Visual metrics and test sets
S4 Jimenez et al. (2023), SWE-bench	Real-world issue resolution evaluation	Repositories and multi-file edits are difficult	Executable integration tests
S5 Aleithan et al. (2024), SWE-Bench+	Critical benchmark data review	Weak tests can overstate success	Independent test quality review
S6 Yu et al. (2024), HumanEval Pro	Harder compositional coding tasks	Simple code benchmarks may be misleading	Multi-step functional tests
S7 Madaan et al. (2023), Self-Refine	Iterative feedback/refinement	Not directly a web-builder evaluation	Chat iteration strategy
S8 Chen et al. (2024),	Targeted iterative	Mathematical	Targeted revision

ID / Reference	Approach / Findings	Critical Limitation	Relevance to draWeb
MAgICoRe	reasoning refinement	reasoning setting, indirect transfer	hypothesis

3.4. Applied and Product Sources (Four Vendor Documents)

The official Figma Make help page [P1], Framer comparison material [P2], the vendor-authored Lovable comparison of Lovable/Bolt/v0 [P3], and Figma MCP help material [P4] document claimed product capabilities. These four sources are vendor documentation rather than independent academic studies; their claims require hands-on confirmation. Eight scholarly research works [S1–S8] are separately listed. arXiv availability alone does not mean a paper is peer reviewed.

3.5. Literature-Driven Requirements

- Capture image/drawn and structured-block inputs as distinguishable formats.
- Evaluate visual fidelity of generated pages against the input design.
- Measure whether exported projects build and run successfully.
- Check generated API endpoints, data persistence, authentication and error cases when supported.
- Keep human-reviewable generated source and version history.
- Test whether natural-language modifications preserve existing functionality.

4. Existing System Comparison

4.1. Comparative Method

Five tools are selected because they cover canvas-based visual building, design-to-code handoff, prompt-to-app creation and full-stack AI generation. The 8 criteria are: initial input, visual editing, AI-assisted generation, frontend output, backend

workflow, code accessibility/export, preview/publishing, and key positioning. «Not verified» must not be interpreted as «not supported».

System	Initial Input / Editing	Code and Backend	Evidence-Backed Distinction
Figma Make	Prompts, frames/images; chat and direct preview/code editing	Generates app code; backend workflows are documented	Design-to-app prototype workflow [P1]
Framer	Visual canvas plus AI website workflows	Website-focused; custom code components and CMS; full backend export not established here	Design-first website production [P2]
Lovable	Natural-language prompts and visual edits	Full-stack with hosted/connected backend workflows and source ownership described	AI-first full-stack app generation [P3]
Bolt.new	Prompt plus browser development environment	Full-stack application coding; live preview and code editor described	Browser-based code-first building [P3]
v0	Text/screenshots and code-oriented refinement	UI and Next.js-oriented full-stack capability; database integrations	Code-first UI/full-stack workflow [P3]
draWeb (planned)	Freehand drawing + drag-drop + uploaded assets + AI chat	Target: exported frontend + selected backend code, preview and ZIP	Distinct intended emphasis: sketch/block-driven workflow; not yet validated

4.2. Comparison Against Consistent Criteria

The table above summarizes technical positioning; the checklist below should be completed with direct, dated hands-on evidence before definitive capability scores or «unique feature» claims are made.

Criterion	Measurement for Each Competing System
C1 Sketch input	Can a freehand drawing be used directly? Test method and record screenshot.
C2 Drag/drop	Can non-developers place and rearrange components?
C3 Imported assets	What file types and restrictions are supported?
C4 AI iteration	Can instructions modify an existing project?
C5 Frontend source	Can the implementation be inspected and exported?
C6 Backend output	Does it generate executable endpoints/data logic or only integrate services?
C7 Preview/export	Is there a live preview and complete local project export?
C8 Reproducibility	Can exported code run without proprietary hosted dependencies?

4.3. Proposed Advantage and Risk

draWeb will be evaluated on the complete user journey: sketch or blocks → structured specification → generated application → AI revision → preview → portable ZIP. Some existing products already support multiple portions of this flow, so novelty should be framed as a focused implementation, usability and reproducibility goal rather than market exclusivity. Major risks: misinterpreted sketches, fragile generated backend logic, insecure code, high inference cost and limited project scope.

5. Technology Comparison

5.1. Decision Criteria

The team has named Vue, Node.js/JavaScript, Go, PostgreSQL, Supabase Auth, OpenAI API, Docker and VPC. These are candidate architecture choices, not evidence that the system currently runs on all of them. Compare alternatives using

implementation effort, runtime suitability, maintainability, reliability, security, integration effort and operating cost.

Decision / Alternatives	Assessment	Provisional Choice / Rationale
Backend services: Node.js / Go / Python FastAPI	Node.js shares JS ecosystem; Go offers compiled services/concurrency; FastAPI simplifies Python AI workflows. Multiple runtimes raise deployment complexity.	Node.js for primary web/API orchestration. Add Go only if a distinct measured worker/service use case emerges.
Database: PostgreSQL / MySQL / MongoDB	PostgreSQL and MySQL provide relational constraints/transactions; MongoDB provides document flexibility. Projects/users/versions form related entities.	PostgreSQL for structured project and version metadata; evaluate object storage for uploaded assets.
AI: OpenAI API / Gemini API / local open-weight model	Hosted APIs shorten prototyping but incur usage costs/data considerations; local models increase hosting burden and may differ in multimodal/code quality.	OpenAI API as initial test candidate, with a replaceable provider adapter and measured quality/cost.
Frontend: Vue / React / Svelte	All support component-driven UIs. Vue matches the team's declared preference; editor integration and library availability must be tested.	Vue as team-proposed framework; canvas library choice remains open.

5.2. Recommended Provisional Architecture

Vue client: canvas/editor, block palette, uploaded assets, chat and preview. Node.js application API: authenticated project CRUD, normalization of canvas data into structured schema, prompt orchestration and export job dispatch. AI provider adapter: sketch interpretation, frontend/backend file generation and modification requests. PostgreSQL: users' project metadata, versions and job state (with Supabase Auth as proposed identity solution). Isolated worker: build/test/export ZIP; Docker containerization with strict sandbox isolation for untrusted generated code. Go may be reserved for a separately justified worker, not introduced solely to increase technology

count. VPC denotes a prospective deployment network boundary rather than a programming framework.

5.3. Validation Plan Before Final Selection

- Prototype the same canvas-to-Vue screen with at least two candidate UI libraries.
- Run one representative CRUD generation prompt through candidate AI providers; measure build success, functional tests, latency and approximate cost.
- Create a small PostgreSQL schema for projects, revisions and assets; test version retrieval.
- Test a Docker-contained generation/build job with restricted permissions, no embedded secrets and resource limits.
- Use a short architecture decision record (ADR) for each selected technology, with evidence and tradeoffs.

5.4. Decisions Needing Supervisor Approval

Confirm supported first-version backend features (for example CRUD and authentication), code export runtime/stack, data retention policy, deployment target, whether Go has a concrete role, and acceptable AI API cost. Until these decisions are made, this document records a justified proposal rather than a final implemented architecture.

Evidence boundary: Bibliographic details, titles, dates, and high-level findings of [S1–S8] were checked against their public arXiv records on 19 October 2026. Claims concern the cited research, not measured draWeb performance. S1 is a foundational 2017 study; S7 is from 2023; S8 addresses mathematical reasoning, so its implications for website editing remain hypothetical. Source-level analysis of full

texts and empirical testing must be completed by the team before a final thesis submission.

References

[S1] Beltramelli, T. (2017). pix2code: Generating Code from a Graphical User Interface Screenshot. arXiv:1705.07962.

[S2] Si, C., et al. (2024). Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering. arXiv:2403.03163.

[S3] Lai, P., et al. (2025). WebRenderBench: Enhancing Web Interface Generation through Layout-Style Consistency and Reinforcement Learning. arXiv:2510.04097.

[S4] Jimenez, C. E., et al. (2023). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770.

[S5] Aleithan, R., et al. (2024). SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992.

[S6] Yu, Z., et al. (2024). HumanEval Pro and MBPP Pro: Evaluating Large Language Models on Self-invoking Code Generation. arXiv:2412.21199.

[S7] Madaan, A., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651.

[S8] Chen, J. C.-Y., et al. (2024). MAgICoRe: Multi-Agent, Iterative, Coarse-to-Fine Refinement for Reasoning. arXiv:2409.12147.

[P1] Figma Help Center. Create a Figma Make file.

[P2] Framer. Compare AI and visual website builders.

[P3] Lovable. Lovable vs Bolt vs v0: AI Builder Comparison.

[P4] Figma Help Center. Tools and example prompts (Figma MCP).

6. Requirements Specification

6.1 Purpose, scope and status

draWeb is intended as a general-purpose AI-assisted platform for creating websites of different categories, including informational, educational, portfolio, blog and e-commerce websites. A user may draw a design on a canvas, assemble reusable blocks, upload media and request generation or modifications in natural language. The intended deliverable is editable frontend and backend source code with a preview and ZIP export.

Scope distinction: support for diverse website categories is a product vision, NOT a verified claim that all possible website types and integrations can currently be generated. The initial diploma demonstration will evaluate representative scenarios with defined features and constraints. The team reports a partially functional UI prototype; individual capabilities below must be implemented and tested before they can be described as complete.

6.2 Stakeholders and user roles

Role	Responsibilities / permitted actions
Visitor (guest)	View platform information; optionally explore a public demo without persistent storage, if implemented.
Registered user	Create and manage projects; use visual editor and AI actions; preview, revise and export generated projects.
Development team / supervisor	Review requirements, implementation milestones, traceability and validation evidence; not necessarily an application role.

6.3 Priority scheme and acceptance methodology

Must = required for the targeted diploma demonstration; Should = important but may follow the initial release; Could = optional future extension. Each acceptance criterion below is a planned verification condition, not a completed test result.

Acceptance is recorded as Pass / Fail / Not tested with screenshots, logs and repository references.

6.4 Functional requirements

ID	Functional requirement	Priority	Acceptance criteria (planned)
FR-01	User authentication	Must	A user can sign up and sign in via Supabase Auth; invalid credentials are rejected and sessions are handled securely.
FR-02	Project creation and persistence	Must	Create a project, save edits, sign out/in and reopen it with the same saved design state.
FR-03	Freehand visual sketching	Must	Draw a simple layout using mouse/pointer input; the sketch remains editable and can be submitted to the generation flow.
FR-04	Drag-and-drop component builder	Must	Add and reposition text, image, button and container blocks; their order and properties persist on save.
FR-05	Media upload	Should	Upload supported image formats with file-size/type validation; selected media appears in the canvas and preview.
FR-06	AI interpretation and frontend generation	Must	Generate a previewable responsive web page from a selected sample sketch/layout; export includes runnable frontend source code.
FR-07	Backend code generation	Must	For at least one explicitly supported use case, generate runnable server endpoints and a documented data model; verify one successful request and one invalid request.
FR-08	Natural-language site modification	Must	For a predefined change (e.g., change button style or add a section), the new preview reflects the instruction without

ID	Functional requirement	Priority	Acceptance criteria (planned)
			manually editing source files.
FR-09	Live preview	Must	User can view generated page and refresh preview following a successful change; failures are visible rather than silently ignored.
FR-10	Download source as ZIP	Must	Download a ZIP containing frontend/backend files and a README with commands and environment-variable placeholders; a reviewer can extract it.
FR-11	Responsive rendering	Must	A representative generated page displays usable layouts in desktop and mobile viewport widths, with no horizontal clipping of primary controls.
FR-12	Version history	Should	Save at least two revisions of a project and restore an earlier saved revision without affecting unrelated projects.
FR-13	AI assistant conversation	Should	User can submit iterative instructions and see request/result history linked to the project.
FR-14	Multi-category templates	Should	Offer starting templates for at least two categories, such as business landing page and simple catalog, with editable content.
FR-15	One-click publishing / hosted URL	Could	If implemented, the deployed site has an accessible URL and a documented way to disable it; otherwise explicitly excluded from first release.
FR-16	Generated e-commerce workflows	Should	In a scoped demonstration, a catalog, account flow or order workflow works using defined endpoints; no assumption of payment gateway or production readiness.

6.5 Non-functional requirements

ID	Quality attribute	Priority	Verification / proposed acceptance target
NFR-01	Usability	Must	In a moderated test, at least 3 of 5 novice participants complete: create a project, edit a component, and find preview/export with minimal assistance. Proposed target; confirm with supervisor.
NFR-02	Security	Must	Authentication uses established provider; secrets stay server-side; access to one user's project by another unauthorised account is rejected; input and uploads are validated.
NFR-03	Reliability and data integrity	Must	A saved project survives browser reload and re-login in all 5 planned test cases; generation failures show an actionable error and do not overwrite the last saved version.
NFR-04	Preview responsiveness	Must	Representative UI tested at 390 px and 1440 px viewport widths; main navigation, CTA and text remain operable.
NFR-05	Performance	Should	For a standard sample project under test conditions, editor initial load <= 5 seconds and non-AI interactions respond within 500 ms at the 95th percentile. Proposed engineering targets.
NFR-06	AI task success	Must	At least 4 of 5 fixed representative design/edit prompts result in a renderable page, with manual scoring of visual alignment and functional behavior. Proposed benchmark.
NFR-07	Portability /	Must	On a documented

ID	Quality attribute	Priority	Verification / proposed acceptance target
	reproducibility		development environment, reviewer can start the demonstration using README instructions and sample configuration without embedded secrets.
NFR-08	Maintainability	Should	Frontend, API, AI orchestration and data-access concerns are separated; critical modules have meaningful tests and documented interfaces.
NFR-09	Accessibility	Should	Key editor actions are keyboard accessible and form fields have labels; basic automated accessibility checks are documented.
NFR-10	Cost and safe execution	Must	AI usage is bounded by request limits; generated code is not executed with unrestricted host privileges; errors and resource limits are documented.

6.6 Example acceptance scenarios

Test ID	Given / When / Then	Linked requirements
AT-01	Given an authenticated user, when a block is added and saved, then the project contains it after logout and login.	FR-01, FR-02, FR-04
AT-02	Given a simple wireframe, when Generate is selected, then a page is rendered and its frontend files can be downloaded.	FR-03, FR-06, FR-09, FR-10
AT-03	Given a generated page, when user requests “Change the main button to blue”, then preview reflects the edit and previous project data is preserved.	FR-08, FR-09, NFR-03
AT-04	Given a predefined catalog scenario, when the generated API receives a valid request, then it returns the expected structured	FR-07, FR-16

Test ID	Given / When / Then	Linked requirements
	response; malformed input is rejected.	
AT-05	Given a sample site, when browser viewport is changed from desktop to mobile, then primary elements stay usable.	FR-11, NFR-04

6.7 Evidence-based justification and traceability

User research observation	Requirement impact	Evidence source
Business users want independent content, catalog and pricing edits.	FR-04, FR-08, FR-16	Chat interviews INT-01 / INT-02
Users want working websites rather than static visual pages.	FR-06, FR-07, FR-10	Chat interviews INT-01 / INT-02; video interview INT-03
Users mention uncertainty about security, quality and ability to inspect the outcome.	NFR-02, NFR-03; preview and source export	INT-01 / INT-02 / INT-03
Novice participant valued simplicity and the ability to understand the prototype quickly.	NFR-01; FR-09	Video interview INT-03
Business participant requested edits through natural-language messages.	FR-08, FR-13	Chat interview INT-02

6.8 Technology constraints and open decisions

Proposed stack from team discussions: Vue.js frontend; JavaScript/Node.js APIs; PostgreSQL; Supabase Auth; OpenAI API; Docker; possible Go service; and VPC networking if deployment warrants it. These are proposed implementation constraints, not evidence of existing integration. Open decisions include Go service responsibility, hosting/deployment, execution sandbox for AI-generated code, target frameworks of generated applications and the first supported backend templates.

6.9 Scope limitations and delivery stages

Stage 1 (diploma verification): visual editor, generation/editing for representative scenarios, runnable code export, project persistence, responsive preview and basic security checks. Stage 2 (potential extension): broader application categories, additional backend modules, deployment automation, advanced templates

and integrations. No guarantee of support for arbitrary websites, advanced payment processing, external provider APIs or production-grade security without dedicated testing.

6.10 Validation and change control

Requirements should be entered as YouTrack tasks with owner, priority, estimated delivery, acceptance checks and links to Git commits / pull requests. Each requirement status must be updated from Planned to Implemented to Verified based on recorded evidence. Supervisor feedback and changes in technical feasibility should trigger documented revisions of this specification.

Document status: Draft requirements specification for supervisor review. Targets and acceptance thresholds are proposed and require agreement; no tests are claimed as passed.

7. Architecture and Diagrams

7.1 System Context

The System Context diagram shows the system as a whole, its user and its external dependencies. The user creates projects, submits prompts, reviews proposed changes and receives source files. An external AI provider processes requests through the server; the API key is never passed to the generated website.

The system boundary includes the UI, API and SQLite. The AI provider is external.

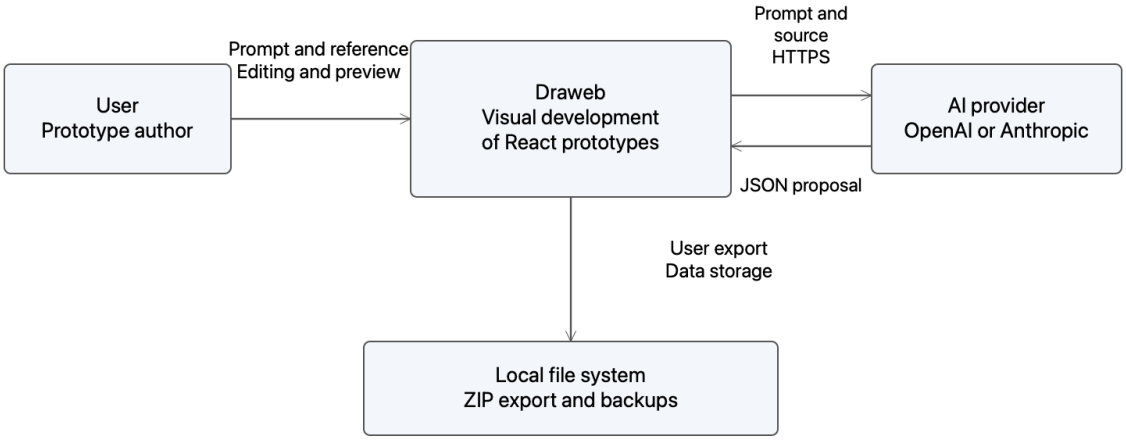


Figure 7.1 — Draweb System Context

The Draweb boundary includes the browser interface, server API, SQLite storage and compilation mechanism. The local file system stores the database and encryption key; the user downloads the ZIP export. OpenAI and Anthropic are alternative external providers.

The current scope covers frontend prototypes using React and React DOM. Backend generation and execution, collaborative editing and website deployment belong to later phases.

7.2 Use Cases

The Use Case Diagram describes user goals. The primary actor is the user; the external supporting actor is the AI provider. All project operations require an authenticated account. A configured provider is required only for generation.

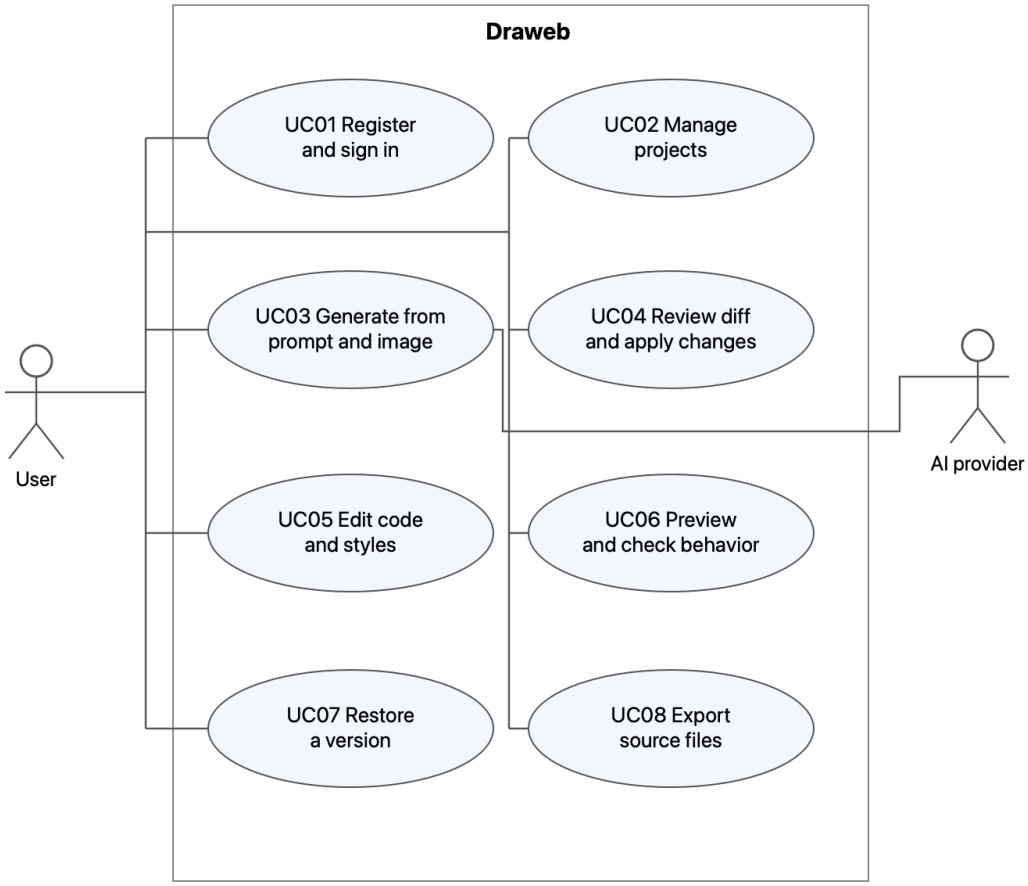


Figure 7.2 — Use Case Diagram

Main flow UC03–UC04: the user opens a project, enters a prompt and optionally attaches an image. The server obtains a proposal from the model, validates the file structure and checks compilation. The user reviews the diff and explicitly applies the changes. Applying a proposal updates the source files, revision and preview.

Alternative flows: the model asks for clarification; the provider returns an error; the proposed code fails to compile; or the project revision has changed. Changes must

not be applied automatically in these cases. Diagram lines indicate actor participation rather than execution order.

7.3 Draft Component Structure

The Component Diagram describes the internal structure of the MVP. The interface calls the Express API; the server checks access and revisions, uses SQLite, invokes the AI adapter and bundles source code for preview. These are logical components of one application rather than separate microservices.

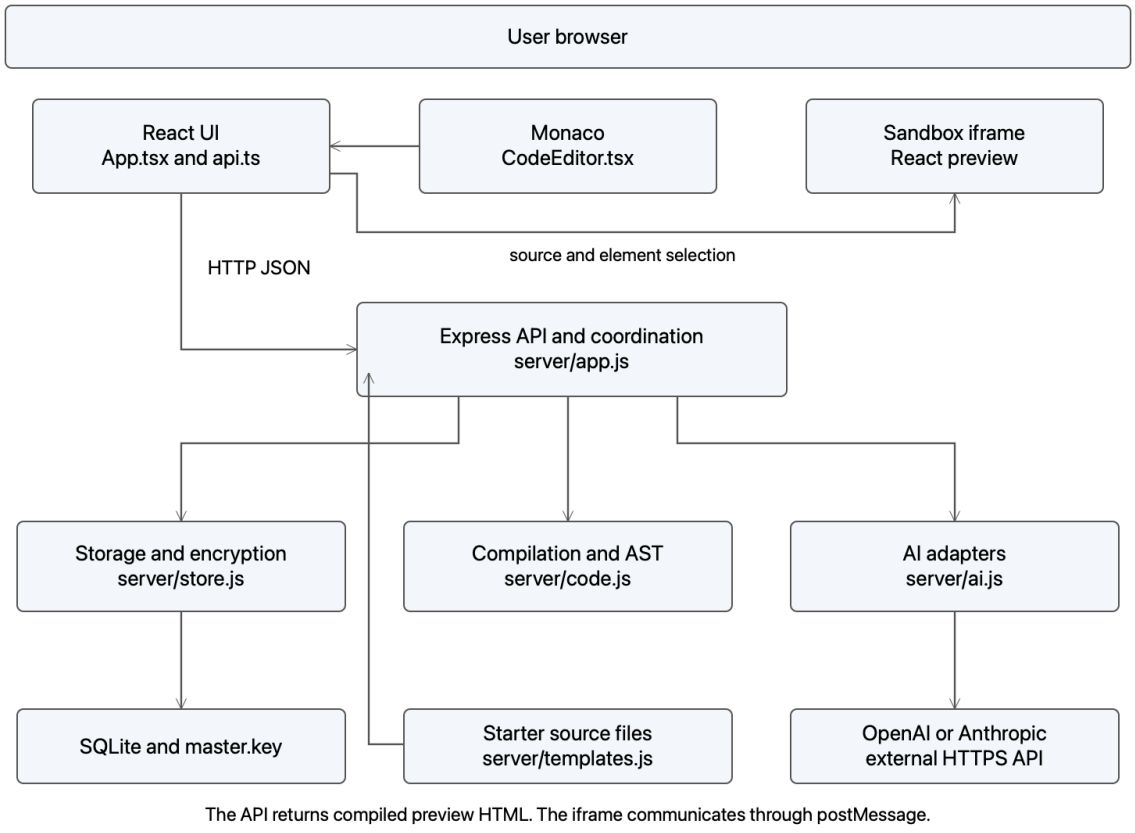


Figure 7.3 — Draft Component Diagram

Responsibilities and Trust Boundaries

App.tsx manages the workspace, api.ts sends requests, and CodeEditor.tsx integrates Monaco. app.js coordinates operations. store.js stores accounts, projects and snapshots, hashes passwords and encrypts keys. ai.js normalizes provider

requests. code.js compiles files in memory and edits JSX through an abstract syntax tree. templates.js contains starter projects.

The preview runs in a sandboxed iframe without same-origin privileges. The Content Security Policy restricts network activity; element-selection messages are checked against their source and channel. This isolates browser privileges but does not limit CPU or memory: an infinite loop can still block the browser process.

8. UX / Prototype

Draweb is a browser-based workspace for creating editable React website prototypes from text prompts and reference images. Users can inspect source code, review AI proposals, refine supported properties visually, preview the result and export the project. The current interface uses the name VisionForge AI.

8.1 Account Registration

A new user creates an account by entering a name, email address and password. The registration screen provides the Create workspace action and a link to the sign-in form for existing users. The interface requires a password of at least ten characters.

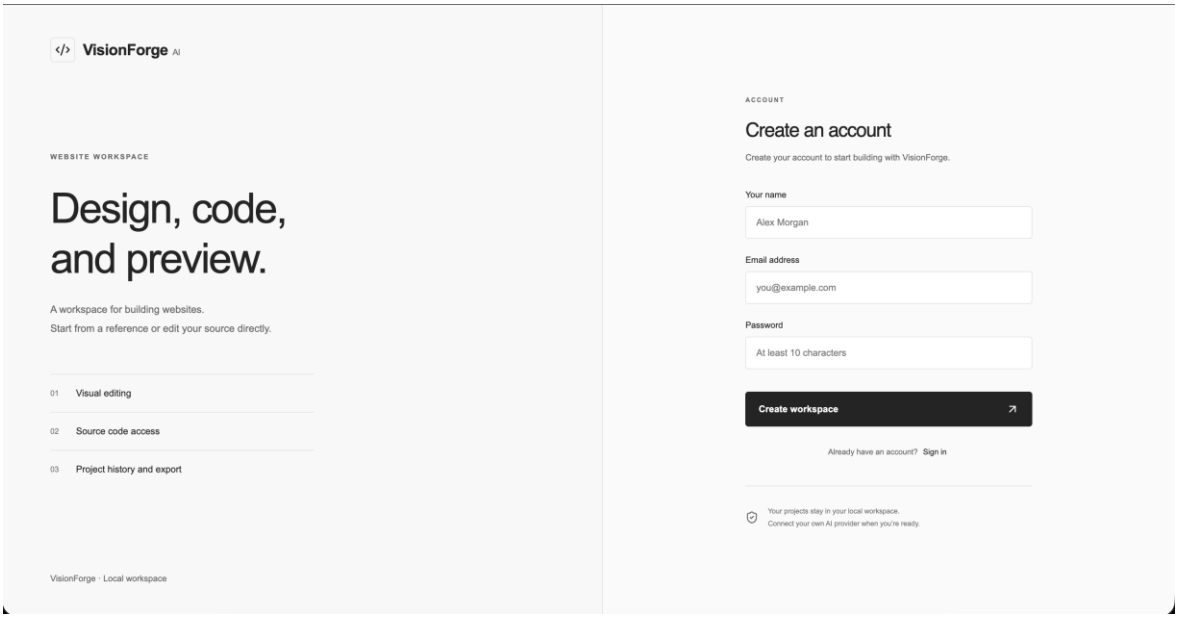


Figure 8.1 — Account registration

After registration, the user enters a personal project workspace. Accounts and sessions are stored in SQLite, so access is managed by the application server rather than by a temporary browser-only profile.

The main purpose of registration is to associate saved projects, AI settings and editing history with an individual account. AI configuration is a separate step; a user can explore a starter project before connecting a provider.

8.2 Sign In and Account Access

Existing users sign in with their email address and password. The Sign in action opens their workspace, where they can continue working on saved projects. The form also provides a Create an account link for users who have not registered.

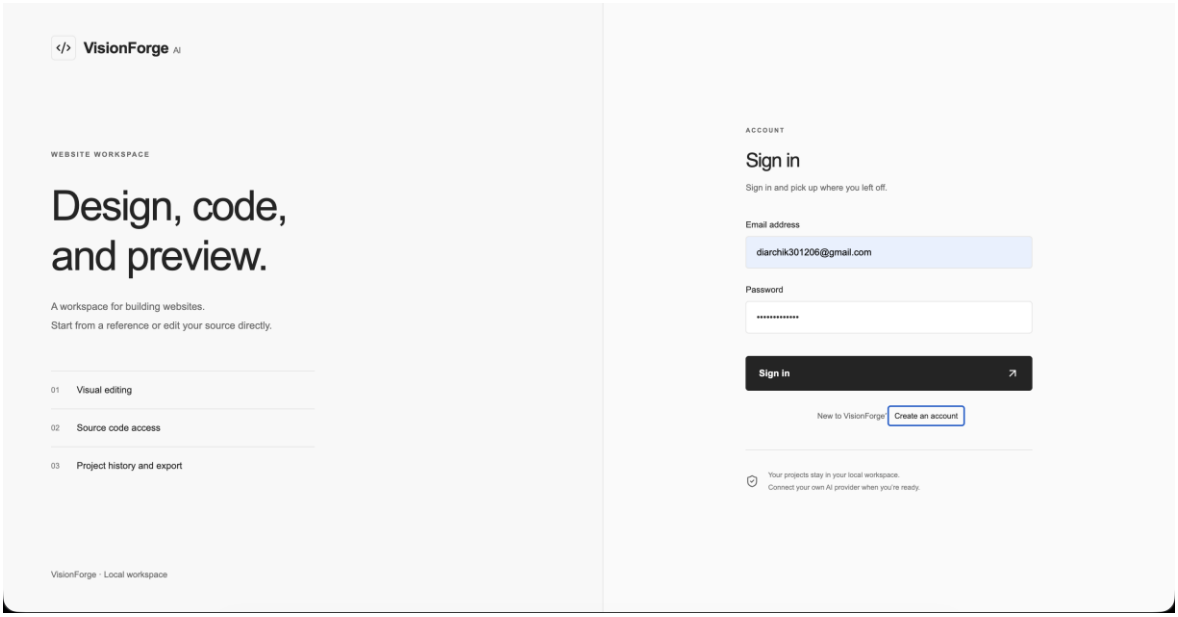


Figure 8.2 — Sign-in screen

Session Management

The application uses expiring HttpOnly sessions. Signing out ends the current session and closes access to protected project operations. The server checks project ownership, so signing in to one account does not grant access to another account’s projects.

Account Security

Passwords are stored as scrypt hashes. AI provider keys are encrypted on the server and are not returned as readable values in the settings response. Generated websites and exported projects do not receive the platform's credentials.

Password reset, email verification and multi-factor authentication are outside the current MVP scope.

8.3 Project Dashboard

The dashboard is the entry point for managing website prototypes. It displays the user's project collection and provides a search field for finding existing projects. An empty workspace explains how to create the first project or open the Atelier starter.

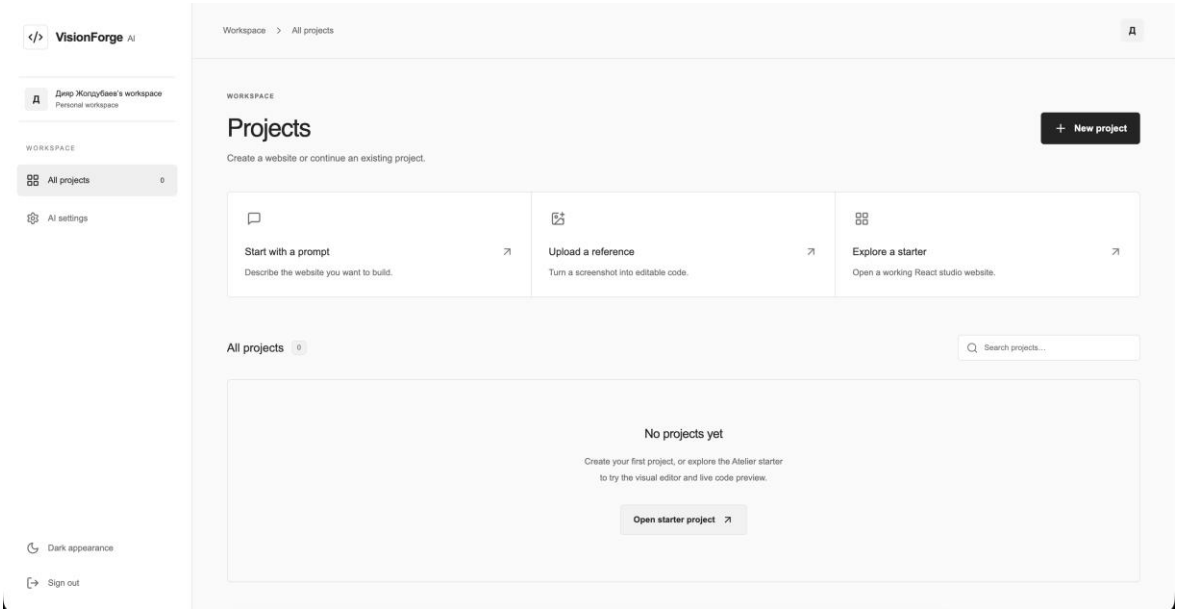


Figure 8.3 — Project dashboard and available starting options

Starting Options

Start with a prompt begins a project that the user can describe in the editor. Upload a reference supports starting from an image of the intended design. Explore a starter opens an existing working React website so that users can try editing and previewing without first generating a page.

Project Management

The implemented project operations include creation, listing, search, renaming, duplication and deletion. Saved projects retain their source files and reference images. The sidebar provides access to AI settings, appearance switching and sign-out.

The project dashboard belongs to the platform interface. Switching its appearance does not change the design or styling of the user's generated website.

8.4 Project Creation

The New project action opens a creation dialog. The user enters a project name and selects Create project. The dialog explains that reference images can be uploaded later in the editor.

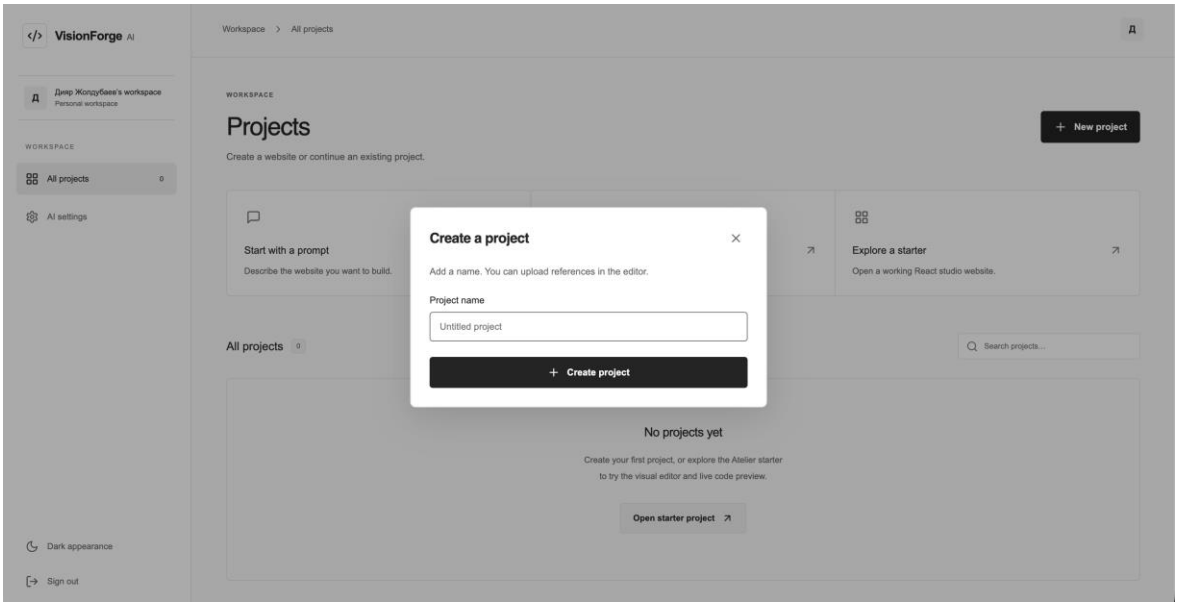


Figure 8.4 — Project creation dialog

Initial Project State

A newly created project opens in the editing workspace with initial React source files. The project explorer displays the file structure and project details, including framework, language, file count and revision. These details help the user identify the current editing state.

Reference Images

The Reference area accepts PNG, JPEG and WebP images up to 5 MB. A reference can guide an AI request together with the text description and current source files. The uploaded image is stored with the project and can be reused in later requests.

An image provides visual guidance, while the prompt should specify intended content and behavior. Uploading a screenshot does not by itself establish how hidden interactions or backend features should work.

8.5 Editing Workspace and Live Preview

The editor combines a project explorer, central preview canvas and right-hand assistance panel. Design and Code modes let users switch between inspecting the page visually and editing its source. Panels can be collapsed to provide more space for the canvas.

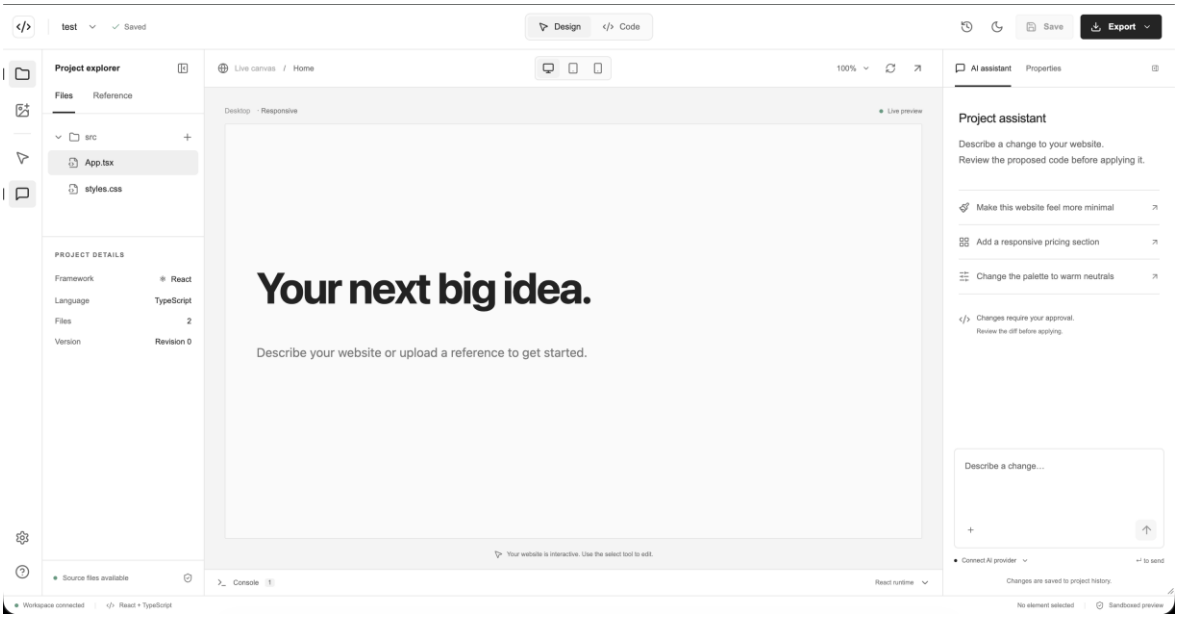


Figure 8.5 — Initial workspace in Design mode

Preview Controls

Desktop, tablet and mobile controls change the preview width. Zoom controls help inspect the canvas, while the separate preview tab provides another way to view

the generated page. The preview executes compiled React code, including local event handlers and links.

Saved State and Feedback

The toolbar indicates whether changes have been saved and provides access to version history and export. Compilation failures appear in the canvas, and runtime errors are reported in the Console. A failed preview does not prevent the user from opening and correcting the source code.

The preview runs in a sandboxed iframe. It supports local React interactions; arbitrary API calls, external scripts and network form submissions are restricted in this phase.

8.6 AI Assistant

The AI assistant allows users to request a new layout or describe changes to an existing website in natural language. Example requests include simplifying the design, adding a responsive pricing section or changing the color palette. The conversation is stored with the project.

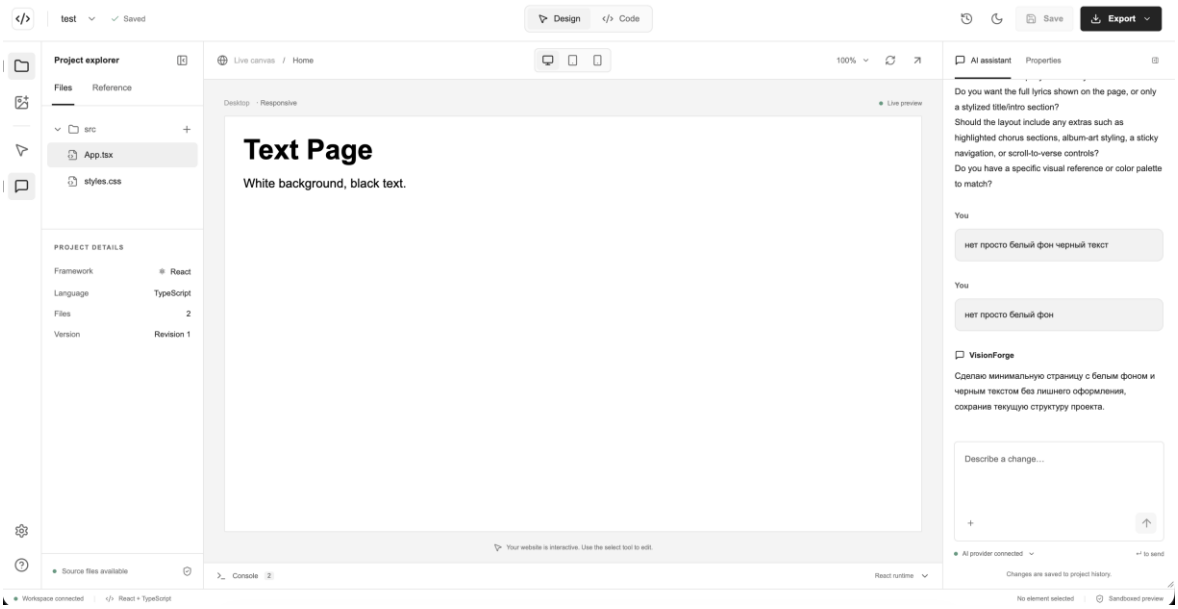


Figure 8.6 — AI conversation alongside the generated page

Provider Configuration

AI settings allow the user to select OpenAI or Anthropic, enter an API key and specify a model identifier. Image-based requests require a model that accepts image input. The chosen model is stored per account rather than fixed in the interface. Provider usage is billed by the selected provider.

Generation Behavior

The server sends the prompt, current source, relevant conversation context and optional reference image to the provider. It validates the returned proposal and checks compilation before presenting file changes for review. If the intended behavior is unclear, the assistant can ask clarification questions instead of generating files.

Provider errors and invalid responses are reported to the user. A generated response remains a proposal until the user chooses to apply it.

8.7 Reviewing and Applying AI Changes

The Review AI changes dialog presents the existing and proposed source side by side. File tabs allow the user to inspect each changed file before making a decision. Added and removed lines are highlighted in the diff viewer.

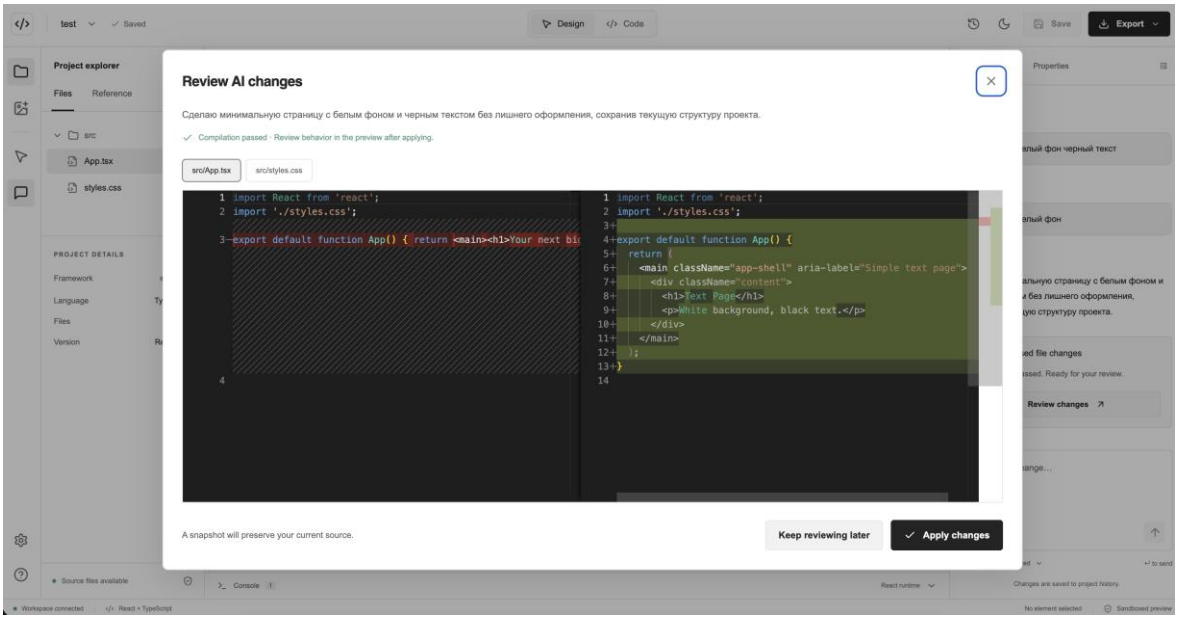


Figure 8.7 — Source diff and explicit approval of an AI proposal

Approval Controls

Apply changes saves the approved source, preserves the previous source in a snapshot and updates the preview. Keep reviewing later allows the user to postpone the decision. A proposal that no longer matches the current project revision is rejected to prevent overwriting newer work.

Compilation and Behavior

The Compilation passed message confirms that the proposed files can be compiled. It does not mean that the page has passed functional tests or that it matches the reference perfectly. After applying a proposal, the user should check the rendered page, responsive layouts, links and interactive elements.

This review step keeps the user in control of changes to the actual project source.

8.8 Source Code Editing

Code mode opens the Monaco editor next to the live preview. The project explorer lets the user select source files such as src/App.tsx and src/styles.css, create additional files or delete supported files. The split view makes it possible to compare source code with its rendered output.

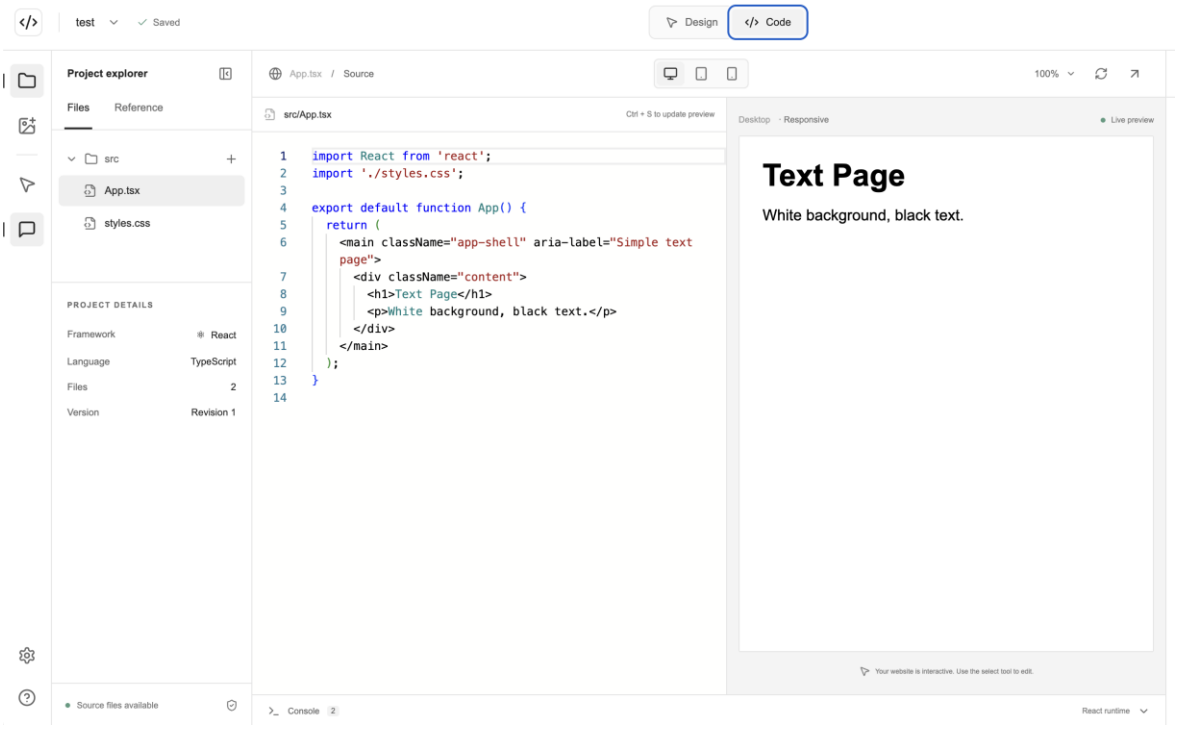


Figure 8.8 — Monaco source editor with a split live preview

Editing Features

The editor provides syntax highlighting, syntax diagnostics, autocomplete, search and formatting commands. Saving with Ctrl/Cmd+S persists the source and rebuilds the preview. Users can therefore modify generated React components and styles directly instead of relying exclusively on AI prompts.

Current Runtime Scope

The Phase 1 runtime supports React, React DOM and relative imports between allowed project source files. Arbitrary third-party package installation and generated backend execution are not available. Full semantic TypeScript diagnostics in the browser editor remain limited until project dependency types are loaded.

8.9 Visual Editing History and Export

Visual Property Editing

In Design mode, the user activates the selection tool and clicks an element in the preview. The Properties panel supports changing static text and properties such as text color, background, size, padding, width and corner radius. Apply to source updates the underlying JSX through an abstract syntax tree.

Visual edits preserve unrelated source and event handlers. The application protects nested text and dynamic style expressions from unsupported changes; those cases can be edited directly in Code mode.

Version History and Persistence

Source changes can be preserved as snapshots. Version history allows the user to restore an earlier source state when a change is unsuitable. Revision checks reject conflicting updates from another tab rather than silently replacing newer data. Projects, messages, settings and snapshots are persisted in SQLite.

Project Export

Export downloads a ZIP archive containing the project source, a Vite package manifest, an HTML entry file, a README, an environment template and a Dockerfile. The user can extract the archive, install its dependencies and run the project independently. The exported archive does not include the platform’s API keys or other credentials.

Typical User Workflow

Create an account and a project, or open a starter. Connect an AI provider when generation is needed. Enter a prompt and optionally upload a reference. Review and apply the proposed diff, refine the result visually or in code, test different preview

widths, then export the source. Restore an earlier version if a later change is unsuitable.

Current Scope and Future Development

The current release is a locally runnable MVP for frontend prototyping. Backend application generation, collaborative editing, GitHub export and deployment integrations are planned for later phases. Local React form behavior must not be presented as a completed backend operation. The application is not yet hardened for public multi-tenant hosting.

Project Reference

Repository: https://github.com/Dirolka/draWeb. Functional scope and operating limits: README.md. The interface illustrations show the supplied application screens.

9. Verification and Evaluation Plan

9.1 Objectives and Test Environment

The Verification Plan defines what is tested, how results are measured and which criteria determine MVP acceptance. It covers the complete user workflow, source integrity, access control and prototype quality. The following sections describe a plan and proposed criteria, rather than completed experimental results.

Test environment: Node.js 24 or later, installed Google Chrome, dependencies pinned by package-lock.json and a separate temporary SQLite database. Record the operating system, CPU, RAM, browser version and tested commit hash before each run. Do not use the production user database.

9.2 Test Matrix

ID and Level	Action or Condition	Expected Result
T01 API	Register, sign in, sign out; request without a session.	A valid session is created. Access ends after sign-out; unauthenticated requests return 401.
T02 API	A second account attempts to open and export another user’s project.	Access is denied without disclosing data; the current API returns 404.
T03 API	Save with a current revision and then with a stale revision.	Current changes are saved; the stale update returns 409 without overwriting data.
T04 AI and API	Submit an image; review and apply a proposal.	The image is included in the request; source files change only after approval.
T05 Negative	Invalid JSON, provider failure or invalid generated code.	An understandable error is shown; project source files remain unchanged.
T06 Compiler	Edit JSX text and styles; attempt to edit dynamic expressions.	Handlers are preserved; unsupported visual edits are rejected.
T07 UI	Edit in Monaco, save and reload the page.	Saved source files are restored; the preview matches the saved version.
T08 UI and API	Restore a snapshot and export a ZIP archive.	Source files match the snapshot; the exported project runs independently.
T09 Security	Read settings, export files and send a request from a foreign Origin.	No plaintext API key is exposed; the forbidden Origin is rejected.

ID and Level	Action or Condition	Expected Result
T10 UI	Test widths of 1440, 768 and 390 px, links and a React form.	Layout, navigation and local event handlers are checked.

Existing automation is available in tests/platform.test.js and scripts/verify-browser.mjs. This matrix extends beyond the existing checks: missing cases must be implemented or executed manually and recorded separately.

9.3 Metrics and Acceptance Criteria

The thresholds below are proposed for the diploma experiment and should be agreed with the supervisor before data collection. A successful build establishes that code compiles; it does not establish correct behavior or visual similarity to a reference.

Metric	Measurement Method	Proposed Criterion
Critical scenarios	Passed critical tests / executed critical tests × 100%. All tests T01–T09 are critical.	100%; no defect involving data loss, key exposure or access to another user’s project.
First generation success	Share of initial responses containing valid files that compile and load without an initial runtime error. Errors and timeouts remain in the denominator.	At least 90% on the fixed task set.
Requirements fulfilled	Share of completed items in the predefined functional checklist for each task.	At least 90% of mandatory items in the final prototype.
Visual similarity	Two independent assessors rate structure, typography, colors, spacing and responsiveness from 1 to 5.	Mean score of at least 4.0; report disagreements between assessors separately.
Task completion time	Time from receiving the task until the checklist is completed, including generation, waiting, edits and retries.	Draweb median no higher than baseline; desired reduction of 20%.
Interface latency	Time from saving source files to a ready preview; 30 runs of the same project after one warm-up run.	p95 of no more than 3 seconds on the recorded test machine; AI latency measured separately.
Integrity and export	Compare source files before and after restoration; inspect ZIP contents and launch the exported project.	Exact match with the expected source files; successful independent launch.

Report median and p95 for time measurements, and numerator and denominator for proportions. Define p95 as the value at rank ceil(0.95 × n) in the sorted sample. Also record AI request counts, provider failures and available token usage information.

9.4 Baseline for Comparison

Baseline Definition

The baseline is conventional React code generation through the same AI API, followed by manual editing in a code editor and execution through Vite. It does not use Draweb visual editing, diff review, automatic application of changes or snapshot history. The baseline must be prepared as a separate experimental workflow.

The comparison evaluates the benefit of the Draweb workflow rather than the superiority of one model over another. Use the same provider, model identifier, references, requirements, initial source files and request limits for both variants. Save the exact system and user messages; document any necessary differences in response format.

Task Set and Experimental Procedure

Prepare 10 tasks: four simple landing pages, three pages with a form or local interactive element, and three multi-section pages with responsive layouts. For each task, save the reference, mandatory elements, functional checks and viewport dimensions in advance. External images and fonts must be equally available to both variants.

Run each task three times in each variant, for 60 attempts in total. Limit each attempt to 20 minutes and five AI requests. Record an incomplete task as a failure and retain its actual elapsed time. Keep API failures and timeouts in the primary dataset and record their causes separately.

Allocate repetitions so that both participants use both variants for each task, changing the order between tasks. Before measurement, complete one practice task per variant outside the dataset. Account for learning effects when interpreting results.

Assessment and Hypothesis

Provide assessors with anonymized screenshots at identical viewport sizes and functional test results. A score of 1 means substantial non-compliance, 3 means partial fulfillment with noticeable differences, and 5 means fulfillment with minimal differences. Use 2 and 4 for intermediate cases.

The primary hypothesis is that Draweb reduces the time needed to obtain an editable prototype without reducing requirements fulfillment. Calculate the time reduction as (baseline median − Draweb median) / baseline median × 100%. Report both medians alongside the success rate, and report successful-attempt times separately.

9.5 Execution and Reporting

Verification Sequence

1. Record the project version, environment, task set, checklists and criteria. Prepare a temporary database and test accounts. Exclude real API keys from reports and screenshots.

2. Run npm ci, npm test and npm run build. Then run npm run test:browser with Chrome installed and dist available. Save logs, dates, exit codes and detected failures.

3. Execute the missing negative and manual checks from the matrix. Check persistence after a server restart and independently launch the ZIP export. Link fixes to test identifiers and repeat affected checks.

4. Conduct the separate experiment with a real provider according to Section 9.4. Controlled responses in automated tests must not be treated as evidence of real generation quality.

5. Summarize results for each variant and list unmet criteria and limitations. Assess whether project objectives were achieved only after collecting measurements.

Record for One Experimental Attempt

Field	Information to Record
Identification	Task, repetition, variant, participant, date and commit hash.
Conditions	Environment, model, parameters, initial files, reference and exact prompts.
Measurements	Time, request count, first generation success and completed checklist items.
Quality	Scores from both assessors for each criterion, screenshots, errors and limitations.
Evidence	Test log, final source files, ZIP archive and location of attempt materials.

Limitations

Two participants and ten tasks provide a preliminary assessment of the MVP. Results depend on participant experience, model version and task selection. Public-service load testing, an independent security audit and usability research with a broader user group require separate work.

Implementation Sources

Repository: https://github.com/Dirolka/draWeb. Scope: README.md. Components: src/App.tsx, src/CodeEditor.tsx, src/api.ts, server/app.js, server/store.js, server/ai.js, server/code.js and server/templates.js. Verification sources: tests/platform.test.js, scripts/verify-browser.mjs and package.json.

10. Project Management and Risks

10.1. Management Approach and Tools

The draWeb engineering team comprises two student contributors: Rassul Tokatov (GitHub account: cybertora; YouTrack username: admin) and Diyar (GitHub account: Dirolka; YouTrack account identifier: 240159). The project utilizes GitHub as the centralized distributed version control system and YouTrack for sprint scheduling, issue assignment, task prioritization, and Kanban-driven workflow tracking. The active codebase encapsulates an emerging full-stack prototype application. Documentation artifacts and source code modifications are maintained independently; completion of a written report section does not automatically denote that the corresponding YouTrack task has satisfied formal review criteria.

Project repository: Dirolka/draWeb (GitHub)

Issue tracker: cybertora.youtrack.cloud (Project: DRA)

The project Kanban board establishes six distinct operational stages: Queue, Development, Review, Test, Test Environment, and Done. The latest telemetry snapshot indicates 2 items in Queue, 9 items in Development, and 0 items in Done. These metrics reflect an interim development snapshot rather than final milestone closure.

10.2. Team Roles and Recorded Contributions

Contributor	**Primary Technical Contribution (Commit Evidence)**	**Commit Reference**
Rassul Tokatov (cybertora)	React/TypeScript frontend configuration; client-side API layer; UI starter templates; source compilation and visual editing engine; Monaco code editor integration; styles and themes; contribution guidelines.	Branch: main (author: cybertora)
Diyar (Dirolka)	Server runtime configuration and dependency setup; entry point architecture and environment examples; SQLite persistence and credentials vault; core API endpoints; AI provider adapter; workspace	Branch: main (author:

Contributor	Primary Technical Contribution (Commit Evidence)	Commit Reference
	engine; platform tests and browser verification.	Dirolka)

The contribution breakdown reflects commit titles documented in repository logs rather than independent runtime benchmarks. Overall team workload is partitioned equally, with individual accountability evidenced by separate user accounts and task ownership.

10.3. YouTrack Issue Register

ID	Issue Title	Assignee	Stage
DRA-1	Prepare Project Proposal	Rassul / admin	Development
DRA-3	Evidence of Relevance	Rassul / admin	Development
DRA-4	Prepare Literature Review	Rassul / admin	Queue
DRA-5	Compare Existing Systems	Rassul / admin	Development
DRA-6	Compare Technologies	Rassul / admin	Development
DRA-7	Define System Requirements	Rassul / admin	Queue
DRA-8	Create Architecture Diagrams	Diyar / 240159	Development
DRA-9	Document UX Prototype	Diyar / 240159	Development
DRA-10	Prepare Verification Plan	Diyar / 240159	Development
DRA-11	Configure GitHub Repository	Diyar / 240159	Development
DRA-12	Document Supervisor Feedback	Rassul / admin	Development

Task DRA-2 was omitted from telemetry logs and is not presumed complete. The register encompasses 11 active issues. All visible tasks carry Serious priority targeting the initial documentation checkpoint on 24 October 2026.

10.4. Milestones, Deliverables and Dependencies

Checkpoint	Expected Deliverable	Dependency / Verification
Initial Research Checkpoint (24 Oct 2026)	Proposal, user interview evidence log, literature review, competitive analysis, technology comparison, and requirements specification.	Reviewed documentation artifacts; active repository links; supervisor approval milestones.
Architecture & Prototype Checkpoint	Context, use-case, and component architecture diagrams; editable source schemas; UX prototype walkthrough and interaction logs.	Synchronized functional requirements; verification of current React/Express/SQLite architecture.
Implementation Checkpoint	Enhanced visual canvas editor, AI code synthesis pipeline, data service integration, and unbundled code export engine.	Formal agreement on supported backend template scope; automated test suite execution.
Validation Checkpoint	Executed test logs, performance telemetry, empirical defect register, and baseline usability comparisons.	Frozen build candidate; reproducible environment; predetermined acceptance criteria.
Submission Checkpoint	Consolidated project dissertation report, audit evidence links, and an immutable Git release tag.	Full alignment of uploaded materials with the tagged repository commit.

10.5. Risk Register

Risk	Impact	Mitigation / Response
AI-generated code failure or insecurity	High	Enforce structural AST validation; compile and run integration tests prior to file application; require explicit user review; isolate API credentials.
Scope creep beyond MVP boundaries	High	Strictly decouple current React prototype scope from unconstrained full-stack vision; limit validation to a predefined set of starter templates.

Risk	Impact	Mitigation / Response
Divergence between specification and codebase	High	Reconcile early architectural proposals (Vue/Go/PostgreSQL) with the active React/Express/SQLite codebase; align all written documentation.
Unauthorized access or credential exposure	High	Implement role-based authorization tests; encrypt stored credentials; enforce server-only environment variables; audit configuration samples.
Schedule slippage or ambiguous ownership	Medium	Assign single designated owner per task with testable acceptance criteria; execute weekly Kanban reviews; immediately flag blocked dependencies.
Missing reproducibility or audit evidence	High	Publish an immutable prob-period-1 Git release tag; retain editable diagram source files, automated test logs, and author-tagged commit histories.
API cost overruns or provider outages	Medium	Enforce token consumption ceilings and rate limits; log network exceptions; implement non-destructive local fallback states.

10.6. Traceability and Review Rules

For each technical implementation task, the designated student engineer must specify the target output, objective acceptance criteria, milestone deadline, and the corresponding GitHub branch, commit hash, or pull request. Task identifiers must be included in commit headers (e.g., DRA-8-architecture-diagrams). All code modifications require review by the peer team member prior to merging into the main branch.

Documented repository commits (sample short hashes):

- **cybertora:** 9cd24ac, 1e9e4d6, cbca3db, 3f7806d, cc9afd4, d29a072, baa2d54
- **Dirolka:** c3b4cc3, aae815c, 1e89466, b4ff89a, 42a0074, b2a25e6, 0fe4f6b

10.7. Current Status and Next Actions

At the current review checkpoint, the project maintains an active prototype codebase, bifurcated Git commit histories, draft theoretical and specification documents, and an operational Kanban board. Zero issues have been moved to the

Done column; academic supervisor sign-off and autonomous full-stack generation remain pending verification. Immediate priorities include: linking every tracker issue to concrete deliverables, committing editable architecture diagram sources, ensuring technical stack consistency across all sections, executing formal verification suites, and publishing the consolidated report alongside the immutable Git tag prob-period-1.

Evidence limitations: This section is compiled from team repository logs and project tracker telemetry. Account permissions, individual task criteria, runtime execution, and release tag creation must be formally audited prior to final thesis submission.

11. Supervisor Feedback

11.1 Feedback Received

During the project discussion, the supervisor emphasized the importance of introducing unique features that would differentiate draWeb from existing AI-powered website builders.

The supervisor recommended that the team focus on developing distinctive functionality rather than simply reproducing the capabilities of existing platforms.

11.2 Team Response

In response to this feedback, the development team identified two proposed distinguishing features:

1. Freehand Sketch-to-Full-Stack Generation

Users will be able to draw website layouts directly on a digital canvas. The AI system is intended to interpret these sketches and generate the corresponding frontend and backend code.

2. AI-Powered Visual and Functional Editing

Users will be able to modify both the visual design and the functionality of generated websites using natural-language instructions and visual editing tools, without manually writing code.

11.3 Implementation Status

Both features are included in the planned functionality of draWeb. Their technical implementation and evaluation will take place during subsequent development stages.

11.4 Conclusion

The supervisor's feedback was taken into consideration during the project's feature-planning stage. The team will focus on implementing and evaluating the proposed distinguishing features to demonstrate the project's practical value and differentiation from existing solutions.